

Trabajo Práctico N°2: Sistema de Gestión de Envíos y Distribución

Introducción

En el Trabajo Práctico N.º 1 desarrollamos una primera versión de un sistema para una empresa de logística.

En esa primera versión modelamos los elementos básicos de la operación:

- los **paquetes** que son transportados;
- los **envíos** que representan las operaciones logísticas;
- y una clase **App(main)** encargada de crear objetos, relacionarlos y probar el funcionamiento del sistema.

Durante el desarrollo trabajamos con clases, objetos e instancias, atributos, métodos, constructores, encapsulamiento, modificadores de acceso, getters y setters, estado de los objetos, sobrecarga de constructores, mensajes entre objetos y listas.

El sistema permite que un **Envío** trabaje con uno o más **Paquete**, que los paquetes cambien de estado y que la aplicación gestione múltiples envíos.

Sin embargo, la empresa comenzó a utilizar el sistema y detectó que el modelo actual resulta demasiado sencillo para representar la operación real.

Una nueva etapa para el sistema

La empresa ahora necesita representar **quién solicita los envíos, dónde son procesados y cómo se puede reconstruir el recorrido de cada envío.**

Además, algunas operaciones que anteriormente podían realizarse de manera directa ahora deben respetar reglas del negocio.

Por este motivo, deberán **evolucionar el diseño del TP1.**

No se trata de comenzar un proyecto nuevo.

El objetivo es tomar el sistema existente, analizar las nuevas necesidades y decidir qué objetos, responsabilidades y colaboraciones deben incorporarse.

No se proporcionará una lista completa de atributos y métodos.

Parte del trabajo consiste en **analizar el problema y decidir qué información debe conocer cada objeto y qué responsabilidades debería tener**.

Objetivos

Al finalizar este trabajo, el sistema deberá permitir:

- representar clientes que solicitan envíos;
- representar sucursales que participan en el recorrido de los envíos;
- registrar y consultar múltiples envíos;
- registrar el recorrido de un envío;
- mantener un historial de movimientos;
- realizar consultas sobre los envíos;
- impedir operaciones que no sean válidas;
- validar correctamente los cambios de estado;
- calcular información general sobre los envíos registrados;
- y mantener una adecuada distribución de responsabilidades entre los objetos.

Nivel 1 — Clientes, sucursales y gestión de envíos

La empresa necesita incorporar información que anteriormente no era necesaria.

Nueva necesidad: Clientes

Hasta ahora, el sistema sabía qué paquete se transportaba, pero no quién había solicitado el envío.

A partir de ahora, cada envío debe estar asociado a un **cliente**.

Un cliente puede realizar varios envíos a lo largo del tiempo.

Analicen

¿Qué información mínima necesita el sistema para identificar a un cliente?

¿Qué información debería conocer un **Cliente**?

¿Tiene sentido que un cliente pueda tener varios envíos asociados?

¿Cómo representarían esa relación?

Preguntas de ayuda

- ¿Un cliente debería guardar un único envío o varios?
- Si puede realizar muchos envíos, ¿qué estructura de datos necesitan?
- ¿Quién debería encargarse de registrar que un cliente realizó un nuevo envío?
- ¿Debería **App** modificar directamente la lista de envíos del cliente?
- ¿Qué responsabilidad debería tener el propio **Cliente**?

Pista

Si un objeto puede tener una cantidad variable de otros objetos asociados, probablemente necesiten pensar en una:

List<Envío>

Pero no agreguen una lista simplemente porque "hay que usar listas".

Pregúntense primero:

¿Qué representa esa lista dentro del problema?

Nueva necesidad: Sucursales

La empresa posee distintas sucursales.

Un envío puede comenzar su recorrido en una sucursal y posteriormente pasar por otras antes de llegar a su destino.

Por ejemplo:

Sucursal Viedma



Sucursal Bahía Blanca



Sucursal Buenos Aires



Destino

El sistema debe poder registrar qué sucursal está procesando un envío.

Analicen

¿Qué debería conocer una **Sucursal**?

¿Qué operaciones realiza una sucursal sobre un envío?

¿Una sucursal debería conocer un único envío o varios?

¿Qué sucede cuando una sucursal recibe un envío?

¿Qué sucede cuando lo despacha?

Colaboración entre objetos

A partir de este punto, el sistema deberá tener situaciones donde varios objetos colaboren para resolver una operación.

Por ejemplo, imaginemos que una sucursal recibe un envío.

Una posible interacción sería:

App

↓

Sucursal.recibirEnvio(envio)

↓

Envio.recibir()

↓

Paquete.recibir()

No necesariamente deben implementar exactamente esta solución.

La consigna es que analicen:

¿Qué objeto recibe el mensaje inicial?

¿Qué objeto tiene la responsabilidad de realizar cada acción?

¿Qué objetos necesitan colaborar?

Pregunta importante

Supongamos que queremos despachar un envío.

Una posibilidad sería:

```
envio.getPaquete().setEstado(...);
```

Pero:

¿Es realmente responsabilidad de **App** decidir cómo cambia el estado de un paquete?

Orientación

No debería ser necesario que un objeto externo conozca todos los detalles internos de otro objeto.

Una alternativa más orientada a objetos sería enviar un mensaje:

```
envio.despachar();
```

y permitir que **Envio** coordine las acciones que correspondan.

Por ejemplo:

Sucursal



```
despachar(envio)
```

Envio



```
despachar()
```

Paquete



```
iniciarViaje()
```

La implementación concreta queda a criterio del alumno, pero deberán poder justificar por qué asignaron cada responsabilidad de esa manera.

Nivel 2 — Seguimiento e historial

La empresa detectó otro problema.

Actualmente podemos conocer el estado actual de un paquete, pero no podemos saber **cómo llegó hasta ese estado**.

Por ejemplo, si un cliente pregunta:

"¿Por qué sucursales pasó mi envío?"

el sistema no puede responderlo.

Se necesita incorporar un registro del recorrido.

Registro de seguimiento

Cada vez que ocurre un acontecimiento importante en el recorrido de un envío, el sistema debe conservar información sobre lo ocurrido.

Por ejemplo:

Envío recibido en Sucursal Viedma

Envío despachado desde Sucursal Viedma

Envío recibido en Sucursal Bahía Blanca

Envío despachado desde Sucursal Bahía Blanca

Envío entregado

Para representar estos acontecimientos deberán incorporar una nueva clase.

Pueden llamarla, por ejemplo:

Movimiento

o:

RegistroSeguimiento

La elección queda a criterio de cada uno.

Diseñando RegistroSeguimiento

No se proporcionan sus atributos.

Analicen:

¿Qué información necesitaríamos para reconstruir lo que ocurrió con un envío?

Preguntas de ayuda

- ¿Necesitamos saber cuándo ocurrió?
- ¿Necesitamos saber dónde ocurrió?
- ¿Necesitamos saber qué ocurrió?
- ¿Necesitamos relacionar el movimiento con una sucursal?
- ¿Necesitamos relacionarlo directamente con un envío?

No agreguen información solamente porque "parece útil".

Cada atributo debe tener una razón dentro del problema.

Historial de un envío

Un envío puede tener muchos movimientos.

Por lo tanto, deberán decidir cómo representar:

Envío

└─ historial

└─ Movimiento

— Movimiento

— Movimiento

— Movimiento

Pregunta de diseño

¿Quién debería administrar el historial de un envío?

Una posibilidad es que **Envío** sea responsable de sus propios movimientos.

Por ejemplo:

```
envio.registrarMovimiento(...);
```

Pero nuevamente:

No se busca copiar esta solución literalmente. Deberán analizar qué objeto tiene mayor responsabilidad sobre esa información.

Consultas sobre el historial

El sistema debe permitir consultar el recorrido de un envío.

Como mínimo, deberá poder:

- mostrar el historial completo;
- informar por qué sucursales pasó;
- mostrar el último movimiento registrado.

Pregunta de diseño

¿Quién debería responder cuando alguien pregunta por el historial de un envío?

Orientación

Probablemente sea más apropiado pedirle al propio envío:

envio.mostrarHistorial();

que extraer todos sus movimientos desde App y recorrerlos manualmente.

La idea es aplicar el principio:

Preguntar por comportamiento antes que pedir datos internos para resolverlo desde afuera.

Ejemplo de colaboración

Una situación posible:

Un envío llega a una sucursal.

Podría producirse una cadena de mensajes similar a:

Sucursal

```
|
| recibir(envio)
```

↓

Envio

```
|
| — actualizar estado
|
| — registrar movimiento
```

↓

RegistroSeguimiento

No es necesario que todos estos métodos sean públicos.

Pregúntense:

¿Qué comportamiento necesita estar disponible para otros objetos y cuál debería permanecer oculto dentro del objeto?

Nivel 3 — Reglas de negocio y validaciones

En las primeras versiones del sistema, algunas operaciones podían realizarse sin demasiadas restricciones.

A medida que la empresa comenzó a utilizarlo, aparecieron errores.

Por ejemplo:

- intentar entregar un paquete que todavía no fue despachado;
- intentar despachar un envío que ya fue entregado;
- registrar un envío sin paquetes;
- registrar un movimiento incoherente;
- intentar recibir un envío que ya se encuentra en otra etapa;
- realizar operaciones sobre un envío inexistente.

A partir de ahora, el sistema deberá **impedir estas situaciones**.

Validación de transiciones

El estado de un paquete no puede cambiar arbitrariamente.

Por ejemplo, una secuencia válida podría ser:

RECIBIDO



EN_PREPARACION



EN_VIAJE



ENTREGADO

Pero deberían impedirse situaciones como:

ENTREGADO → EN_VIAJE

o:

RECIBIDO → ENTREGADO

si la lógica definida por el grupo no permite esa transición.

Preguntas de ayuda

- ¿Dónde debería validarse que una transición sea válida?
- ¿Quién conoce el estado actual?
- ¿Quién debería decidir si puede cambiar?
- ¿Conviene que **App** tenga muchos **if** para validar estados?

Orientación

La validación debería estar cerca del objeto que conoce y protege ese estado.

Por ejemplo, en lugar de:

```
if (paquete.getEstado() == ...)
```

desde **App**, puede resultar más adecuado:

```
paquete.entregar();
```

y que el propio objeto determine si puede hacerlo.

Métodos privados

A medida que agreguen validaciones, probablemente encuentren lógica que no debería ser utilizada directamente desde otras clases.

Por ejemplo:

```
private boolean puedeSerEntregado() {  
    ...  
}
```

o:

```
private boolean transicionValida(...) {  
    ...  
}
```

Pregunta

¿Por qué este método debería ser **private**?

Orientación

Porque forma parte de la implementación interna del objeto.

Otros objetos necesitan pedir:

```
paquete.entregar();
```

pero no necesitan conocer cómo el paquete decide si puede ser entregado.

Criterio principal del trabajo

No se evaluará solamente que el programa "funcione".

Se evaluará especialmente:

cómo pensaron y distribuyeron las responsabilidades entre los objetos.

Dos soluciones pueden producir el mismo resultado y, sin embargo, una puede tener una mejor distribución de responsabilidades que la otra.

Por eso, durante el desarrollo, ante cada nueva funcionalidad deberán preguntarse:

¿Qué objeto debería ser responsable de esto y por qué?

Ese razonamiento forma parte central del trabajo.

Entrega

El trabajo deberá incluir el código fuente subido a un repositorio online.